

11_Operators_Part2

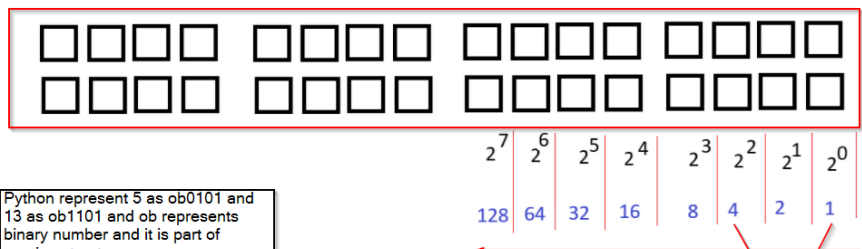
Machine understands and read things in binary 0 and 1

Python uses 32 bits to represent any number

It starts filling things from the last bit to first bit

To represent a number, we dont use all 32 bits.
Just last 4 or 5 or up to what ever binary number
required, we can use to up to that one. see the
examples to undersand on the right bottom

Python represent 5 as ob0101 and
13 as ob1101 and ob represents
binary number and it is part of
naming structure



13 = ????
13 = 8 + 5 = 1101 (binary representation of 13)

$$4 + 1 = 5$$

5 = 4+1, so just replace those
positions 4 and 1 with binary 1
to represents binary form of 5
i.e. 0101

```
a = 5  
bin(5)
```

[17] ✓ 0.2s

... '0b101'

```
print(int('0b1101',base=2))
```

[19] ✓ 0.0s

... 13

```
# Binary Gates
# AND, OR and XOR gates
# A | B | AND | OR | XOR (Inequality Detector)
# 0 | 0 | 0 | 0 | 0
# 0 | 1 | 0 | 1 | 1
# 1 | 0 | 0 | 1 | 1
# 1 | 1 | 1 | 1 | 0
```

[]



```
a=5 #0101 is binary representation
b=3 #0011 is binary representation
print(" a & b results is ", a & b)
print(" a & b binary representation is ", bin(a & b))
print(" a & b value decimal representation is ",int(bin(a&b),base=2))
print(" a|b results is ", a | b )
print(" a|b binary representation is ",bin(a | b))
print(" a|b value decimal representation is ",int(bin(a|b), base=2))
```

[31]

✓ 0.0s

...

```
a & b results is 1
a & b binary representation is 0b1
a & b value decimal representation is 1
a|b results is 7
a|b binary representation is 0b111
a|b value decimal representation is 7
```

```
# Negation (bitwise Not) represented by symbol/character "~"
# Decimal 5 --> represented by 0101 in binary + 1 (0001)
# Result is 0110 which 6 but negation will invert the sign of the give input
# so +5 bitwise not results is -6
# Generic formulae is ~n= -(n+1)
```

[38] ✓ 0.0s

```
a = 5
print(~a)
```

[40] ✓ 0.0s

... -6

```
> ~
for i in range(-10,10,1): # Iterating over -10 to +9 to apply bitwise NOT on each number to see the results
| print("negation of {} is {}".format(i,~i)) #Ignore the syntax as of now as this is for understading bitwise NOT results for each given number
```

[42] ✓ 0.0s

```
... negation of -10 is 9
negation of -9 is 8
negation of -8 is 7
negation of -7 is 6
negation of -6 is 5
negation of -5 is 4
negation of -4 is 3
negation of -3 is 2
negation of -2 is 1
negation of -1 is 0
negation of 0 is -1
negation of 1 is -2
negation of 2 is -3
negation of 3 is -4
negation of 4 is -5
negation of 5 is -6
negation of 6 is -7
negation of 7 is -8
negation of 8 is -9
negation of 9 is -10
```

```
> ~
# Left Shift (bitwise Left << n), here n represents how many times the left shift should happen
a=5 # 5 (decimal) --> 0101 (binary) if we move the values to left then 1010 (binary) = 10 (decimal)
print(a << 1)
```

[48] ✓ 0.0s

... 10

```
# Right Shift (bitwise right >> n), In general one left shift will make value double
# Right shift will make the value into half
b = 6 # 6 (decimal) --> 0110 (binary) if we move the values to the right then 0011 (binary) = 3 (decimal)
print( b >> 1)
```

[47] ✓ 0.0s

... 3